

Access Control Policy

Version: v1.0 ADOPTED **Adopted:** 2026-07-21 by Sami Ben Chaalia (Security Officer) **Date:** 2026-07-21 **Owner:** Security Officer (Sami Ben Chaalia) **Framework mapping:** SOC 2 CC6 (Logical and Physical Access), HIPAA §164.312(a) **Companion evidence:** HIPAA SRA v1 in `compliance/HIPAA_SRA_v1_2026-07-21.md` (railcall-core repo).

1. Purpose

State how RailCall controls who can access RailCall systems and the customer data those systems touch. Written to match the reality of the product today — gaps are called out as gaps, controls that DO exist cite the code that implements them.

2. Scope

This policy applies to:

- **RailCall the operator** (Sami + Pat) — access to the gateway (`railcall-core.onrender.com`), Render dashboard, Stripe dashboard, Postgres, GitHub repositories, and the issuer signing seed.
- **The local RailCall station** deployed on a customer machine — access to the vault, the workflow-editing studio, and the signing seed.

Physical safeguards and customer-side IAM are OUT of scope — those live with the customer or with our hosting providers (Render, Cloudflare).

3. Principles

- **Least privilege.** Every credential — human, agent, or service — receives only the specific scope it needs. Restricted API keys are preferred over full-access ones (the Stripe restricted-key ceremony in the July 2026 paid-tier launch is the canonical example).
- **Local-first by construction.** The station binds `127.0.0.1` only, never `0.0.0.0` (`studio_server.py`). This IS an access control — it removes the entire network attack surface for the studio.
- **Dual control for irreversible actions.** The studio approval token is printed to the launching terminal only, never templated into any served page (`studio_server.py:71-88`). A compromised browser cannot self-approve.

- **Fail closed.** Every gate refuses on unknown input rather than allowing by default. Blind auth on `/v1/seat/checkin` returns 401 on unknown hash; issuer mint returns 503 if the seed is unset; entitlement verification returns free-tier on any tamper.

4. Controls in force today

4.1 §164.312(a)(2)(iv) Encryption of at-rest secrets — IMPLEMENTED

The Ed25519 signing seed for both the station and the CLI is held in the OS keychain — macOS login keychain via `security(1)`, Windows DPAPI, Linux Secret Service via `libsecret`. Implementation in `seed_store.py`; migration from historical plaintext-file storage is idempotent and fail-safe (the keychain write MUST succeed before the plaintext copy is dropped — `test_seed_store.py:5b` guards this).

BYOK credentials for integrations sit in a 0600 vault, AES-256-GCM + scrypt encrypted, opt-in per integration (`vault.py:38–44,95–110`).

Honest limit: the keychain protects against copying, backups, and sync-service leakage. It does NOT stop malware running as the same OS user. Documented in `seed_store.py` `_BACKEND_NOTE` and surfaced via `railcall doctor`.

4.2 §164.312(d) Person/entity authentication — PARTIAL (strong for agents)

- **Agent-to-gateway auth is strong:** mandatory Ed25519 signature verification on every request; no passwords carried across the wire on the paid path (blind sha256(api_key) for `/v1/seat/checkin`); failed verify returns 401 (`primitives/gateway_auth.py:1–30`).
- **Cross-install entitlement replay is prevented:** entitlement tokens are bound to the install pubkey inside the signed body; a copied token activates NOTHING on another machine (`entitlement.py verify_entitlement`).
- **Human authentication on the station itself is a GAP** — see §5.

5. Accepted gaps

5.1 §164.312(a)(2)(i) Unique user identification — GAP

The station is a single-user local runtime. There is no per-user identity; any local process reaching loopback is trusted as the operator. Multi-user hardening is on the product roadmap. For today, mitigation is the customer OS user boundary — a customer deploying to a shared host must rely on their own login controls.

5.2 §164.312(a)(2)(ii) Emergency access procedure — GAP

No break-glass procedure documented in the station. The customer §164.308(a)(6) incident response covers this at the operational layer.

5.3 §164.312(a)(2)(iii) Automatic logoff — GAP

No idle timeout on the studio session. The compensating control is the loopback-only bind — an attacker with local session access on the customer box has broader problems than an unlocked studio. Timeout addition is on the roadmap.

6. Access to RailCall-operator systems

System	Access holders	Grant	Revoke
Render (gateway)	Sami, Pat	Direct env-var edits on the railcall-core service; env groups require explicit service linking	Remove from Render team
Stripe	Sami (admin invited by Pat 2026-07-21), Pat (owner)	Standard keys via account owner; restricted keys per-task	Delete from Stripe team; revoke keys individually
GitHub (patl4588 org)	Sami, Pat	Repo collaborator invite	Remove collaborator
VPS 157.230.177.45 (railcall.ai website)	Sami (sami@ user, <code>~/.ssh/id_ed255199</code>), Pat (root)	SSH key add	Remove authorized_keys entry
Issuer signing seed (paid-tier root)	Sami only	1Password + offline backup	If lost: re-mint keypair, re-pin <code>ISSUER_PUBKEY_HEX</code> , cut new station release

7. Access review

Reviewed at each station release AND at least annually. Any change to the "access holders" column requires a corresponding update here.

8. Related controls

- Encryption of at-rest secrets: measure P0-1 (seed_store integration)
- Audit trail: measure P0-2 (audit_chain), enforces §164.312(b)
- PHI egress: measure P0-3 (phi_guard), enforces §164.312(e)(1) transmission security